# ACADEMIC PROJECT REPORT

## ChainOps: Decentralized Cloud Infrastructure Engine

*Immutable Orchestration, Smart Contracts, Smart Card HSM Custody, and Raft Quorum Consensus on a Physical 3-Node Raspberry Pi Cluster*

**Course Module: BlockChain & Smart Contract**

**Academic Year 2026/2027**


**Authors / Project Contributors:**

**ASLOUNE Yahya** **SALIOUI Ilyes** **SBAI Ahmed Taha**

**Under the Supervision of:**

**Prof. Meryeme AYACHE**

*Professor at the National Institute of Posts and Telecommunications (INPT)*

**Department: Embedded Systems & Digital Services (SESNUM – INE3)**

**National Institute of Posts and Telecommunications (INPT) – Rabat, Morocco**


*« Modern cloud computing infrastructure must no longer rely on blind trust in centralized root administrators, but on immutable cryptographic proofs, physical hardware security tokens, and fault-tolerant distributed consensus. »*

# Contents

**Executive Summary** 1

**Résumé Analytique** 2

**1 Introduction, Context & Problem Statement** 3
- 1.1 Technological Context: Vulnerabilities of Centralized Cloud Systems . . . . 3
- 1.2 The Paradigm: Infrastructure as a Ledger . . . . 3
- 1.3 Academic Objectives & Technical Specifications . . . . 4

**2 Physical Cluster Hardware & Zero-Trust Network Topology** 6
- 2.1 Physical Bare-Metal Raspberry Pi 4 Cluster . . . . 6
- 2.2 End-to-End System Topology . . . . 6
- 2.3 Solving the Kubernetes Control Plane Ingress: Layer 4 TCP Tunneling . . 7

**3 Cryptographic Identity, Smart Cards & Hardware Key Custody** 8
- 3.1 Mathematical Foundations: $secp256k1$ Elliptic Curve Cryptography . . . . 8
- 3.2 Smart Card Architecture & Hardware Key Isolation (Java Card) . . . . 8

**4 The Go Smart Contract Gatekeeper Engine** 10
- 4.1 Deterministic Rule Enforcement (`pkg/contract`) . . . . 10

**5 Distributed Raft Consensus on Physical Raspberry Pis** 11
- 5.1 Formal Raft State Machine Replication (`pkg/consensus`) . . . . 11
- 5.2 Fault Tolerance & Network Partition Scenarios . . . . 11

**6 Immutable Ledger Storage via immudb & Merkle Trees** 13
- 6.1 Cryptographic Hash Chaining . . . . 13
- 6.2 Real-Time Audit Verification & Tamper Detection Workflow . . . . 13

**7 Physical Kubernetes Orchestration via client-go** 14
- 7.1 Manifest Synthesis & Node Affinity Scheduling . . . . 14
- 7.2 Internal Go Package Architecture . . . . 15

**8 Experimental Evaluation & Visual Production Tour** 17
- 8.1 Web3 Authentication Portal . . . . 17
- 8.2 5-Step Consensus Deployment Modal . . . . 17
- 8.3 Live Pod Execution on Physical Worker Node . . . . 18
- 8.4 Physical Node Telemetry & Raft Quorum Health . . . . 19
- 8.5 Immutable Blockchain Explorer & Merkle Root . . . . 20

8.6 Smart Contract Anti-OOM Quota Rejection . . . . . 20
8.7 Physical Master Terminal Verification . . . . . 21
8.8 Cloudflare Zero Trust & Local Development . . . . . 21

**9 API Specification & Automated Cluster Operations 23**
9.1 REST API Endpoint Reference . . . . . 23
9.2 Automated Bare-Metal Deployment . . . . . 23
9.2.1 Master Node Deployment (Raspberry Pi 1) . . . . . 23
9.2.2 Worker Node Deployment (Raspberry Pis 2 & 3) . . . . . 24
9.2.3 Pruning Stale Inactive Nodes . . . . . 24

**10 Conclusion & Future Directions 25**
10.1 Summary of Technical & Academic Contributions . . . . . 25
10.2 Future Research Directions . . . . . 25

# List of Figures

1 Decentralized Web3 authentication supporting MetaMask and Local Operator Keypairs . . . 17
2 Live modal tracking client signing, smart contract validation, Raft consensus, and immudb commit . . . 18
3 Production container `test2` scheduled and executing on physical worker node . . . 18
4 Kubernetes manifest generated by client-go alongside real-time stdout/stderr log stream . . . 19
5 Real-time hardware telemetry and Raft quorum voting parameters across all 3 Raspberry Pis . . . 19
6 immudb ledger explorer detailing block hashes, 3/3 Raft receipts, and Merkle root . . . 20
7 Smart Contract Gatekeeper rejecting an allocation exceeding physical bare-metal RAM capacity . . . 21
8 Physical Master Pi console verifying `kubectl get nodes -o wide` in Ready status . . . 21
9 Network ingress infrastructure and local development environment . . . 22

# List of Tables

1 Architectural comparison between conventional cloud infrastructure and ChainOps . . . 4
2 Physical bare-metal hardware specifications of the Raspberry Pi cluster . . . 6
3 Calibrated operational parameters of the physical Raft consensus protocol 11
4 Complete REST API specification exposed by the unified ChainOps cluster daemon . . . 23

# Executive Summary

Contemporary cloud orchestration infrastructures (typified by hyperscalers such as Amazon Web Services, Google Cloud Platform, and Microsoft Azure) are constructed around centralized trust perimeters. Core control-plane operations—including compute provisioning, memory quota enforcement, network ingress, and administrative access delegation—are recorded in conventional mutable relational databases or unchained key-value engines (e.g., standard Kubernetes `etcd`). In these models, a single compromised root credential or a rogue insider can tamper with cluster configurations, allocate unauthorized compute resources, or alter historical audit logs without triggering mathematical detection.

Developed as the capstone engineering project for the **BlockChain & Smart Contract** course at INPT under the guidance of **Prof. Meryeme AYACHE**, this report introduces **ChainOps** : an enterprise-grade, decentralized cloud orchestration platform deployed across a physical bare-metal cluster of three **Raspberry Pi 4 (ARM64)** nodes. ChainOps revolutionizes cloud lifecycle management by treating every container provisioning request as an immutable financial transaction anchored into a 5-step cryptographic pipeline:

1. **Non-Custodial Cryptographic Client Signing**: Workloads are authorized exclusively via *secp*256$k$1 elliptic curve signatures, supporting both Web3 browser extensions (MetaMask) and physical **Smart Cards / Secure Hardware Modules** (Java Card tokens) executing on-chip APDU commands;

2. **Deterministic Go Smart Contract Gatekeeper**: Enforces cluster-wide physical resource invariants via mathematical identity derivation (`ecrecover`) and hard limits to mathematically prevent Out-Of-Memory (OOM) host kernel panics;

3. **Physical 3-Node Raft Consensus Engine**: Implements majority quorum ($N/2+1 = 2$ nodes) voting across physical Raspberry Pis with 150 ms heartbeats, leader election, and signed ballots to eliminate split-brain hazards;

4. **Cryptographic Ledger via immudb**: Sequentially binds transactions into a SHA-256 Merkle tree state chain, exposing recalculable Merkle roots for real-time auditability and instantaneous tamper detection;

5. **Physical Bare-Metal Kubernetes Orchestration**: Officially drives physical worker Pis via Kubernetes `client-go`, deploying pods with node affinity and automated self-healing runtimes to prevent `CrashLoopBackOff` on headless containers.

To achieve true zero-trust networking without public IP addresses or local port forwarding, inter-node mTLS communication and client traffic are routed through Cloudflare

Zero Trust Layer 4 TCP tunnel streams.

---

## Résumé Analytique

Ce rapport présente **ChainOps**, une plateforme d'orchestration cloud décentralisée nativement intégrée sur un cluster physique bare-metal de 3 Raspberry Pi 4 (ARM64), réalisée dans le cadre du module **BlockChain & Smart Contract** de l'INPT sous la supervision de **Mme Meryeme AYACHE**. L'architecture remplace la confiance implicite des infrastructures centralisées par un pipeline cryptographique déterministe articulé autour de cinq piliers : signature asymétrique cliente ($secp256k1$ et cartes à puce Java Card), validation par Smart Contract en Go avec respect strict des quotas mémoire physiques anti-OOM, consensus distribué Raft à majorité physique (quorum 2/3), persistance immuable dans immudb avec calcul de racines d'arbre de Merkle SHA-256, et ordonnancement physique bare-metal via Kubernetes `client-go`. L'isolation réseau est garantie par des tunnels Cloudflare Zero Trust Layer 4 TCP sans aucune exposition d'adresse IP locale.

# 1 Introduction, Context & Problem Statement

## 1.1 Technological Context: Vulnerabilities of Centralized Cloud Systems

The containerization revolution led by Docker and Kubernetes has standardized software deployment globally. However, modern cluster management architectures retain an architectural flaw rooted in legacy client-server design: **centralized authority and blind database trust**.

In a standard Kubernetes deployment, the entire cluster configuration resides within `etcd`. Although `etcd` uses the Raft protocol to achieve fault tolerance across server replicas, it provides **zero cryptographic immutability guarantees**:

- An operating system user with root permissions on the master host can directly alter binary database keys, reconfiguring container privileges or injecting malicious pods without leaving a tamper-evident audit trail;
- Audit logging mechanisms rely on offloading plaintext JSON logs to external servers (Splunk, ElasticSearch, Datadog), which can be truncated, modified, or wiped by compromised administrative accounts;
- Traditional authentication mechanisms (X.509 client certificates, JWT Bearer tokens, static API keys) authenticate the channel, but do not cryptographically bind the author's identity to the exact binary payload of the deployed container.

## 1.2 The Paradigm: Infrastructure as a Ledger

To address these vulnerabilities, this project establishes the **Infrastructure-as-a-Ledger** paradigm.

**Core Axiom: Infrastructure as a Ledger**

In ChainOps, a cloud infrastructure operation (launching a container, allocating RAM, updating replica counts, or tearing down a service) is strictly defined as an **immutable state transaction**. It must be cryptographically signed by the operator's private key, deterministically validated against mathematical quota rules in a Smart Contract, agreed upon by a physical majority quorum through Raft consensus, committed to a Merkle-tree cryptographic ledger, and only then physically realized by the container runtime.

Table 1 contrasts standard enterprise cloud platforms with the ChainOps DLT architecture.

| Feature Dimension | Conventional Cloud (AWS / GCP / K8s) | ChainOps Architecture (Our Solution) |
|---|---|---|
| **Identity & Access** | Static API keys, session tokens, IAM roles | Asymmetric $secp256k1$ cryptographic signatures (EIP-191) |
| **Hardware Key Custody** | Software secrets, env vars, cloud KMS | Physical Smart Cards (Java Card) & Secure Enclaves |
| **Admission Control** | Mutable webhook scripts (OPA / Gatekeeper) | Deterministic Go Smart Contract with hard hardware invariants |
| **Trust Topology** | Blind trust in centralized root admin | Distributed physical Raft quorum ($N/2+1$ physical Pis) |
| **State Persistence** | Mutable relational DB or unchained key-value | Append-only cryptographic ledger (`immudb`) |
| **Audit Verification** | Centralized, mutable log files | Mathematical SHA-256 Merkle root verification in sub-seconds |
| **Network Ingress** | Public IPs, open ports, NAT traversal | Cloudflare Zero Trust Layer 4 TCP tunnel with zero public IPs |

Table 1: Architectural comparison between conventional cloud infrastructure and ChainOps

## 1.3 Academic Objectives & Technical Specifications

Within the scope of the **BlockChain & Smart Contract** curriculum at INPT, our engineering team was tasked with designing, implementing, and validating an end-to-end bare-metal solution meeting the following specifications:

1. **Bare-Metal Physical Deployment**: Deploy on three physical Raspberry Pi 4 ARM64 single-board computers connected over a Gigabit Ethernet switch;

2. **Strict Zero-Local-IP Dependency**: Enable worker Pis and external web clients to communicate with the master control plane entirely over a public domain managed by Cloudflare Zero Trust;

3. **Physical Smart Card & Web3 Key Custody**: Support non-custodial $secp256k1$ key custody using hardware tokens (Java Card tokens) and browser wallets (MetaMask);

4. **Deterministic Smart Contract Gatekeeper**: Implement an automated resource controller in Go that performs elliptic curve recovery (`ecrecover`) and prevents cluster OOM crashes;

5. **Distributed Raft Consensus Engine**: Build a 3-node physical Raft consensus cluster with heartbeats, election timeouts, and signed quorum voting ballots;

6. **Cryptographic immudb Merkle Tree Ledger**: Persist all state transitions into SHA-256 hash chains with real-time Merkle root generation and automated tamper

detection;

7. **Production Kubernetes Orchestration**: Translate approved ledger transactions into native Kubernetes objects via official `client-go` drivers.

# 2 Physical Cluster Hardware & Zero-Trust Network Topology

## 2.1 Physical Bare-Metal Raspberry Pi 4 Cluster

The platform executes on a dedicated 3-node physical Raspberry Pi 4 bare-metal cluster. Operating directly on bare metal exposes the software stack to realistic physical constraints: hardware CPU throttling, limited physical RAM, and kernel cgroup accounting.

| Node Identifier | Cluster Role | Architecture | Local LAN IP | Hardware Specifications |
|---|---|---|---|---|
| **pi-node-01** | Master (Raft Leader) | ARM64 (BCM2711) | 192.168.1.212 | 4 Cores @ 1.5 GHz, 3,794 MB RAM, Debian 11 |
| **pi-worker-1-afe458a1** | Worker 1 (Raft Follower) | ARM64 (BCM2711) | 192.168.1.213 | 4 Cores @ 1.5 GHz, 3,794 MB RAM, Debian 11 |
| **pi-worker-2-d50675de** | Worker 2 (Raft Follower) | ARM64 (BCM2711) | 192.168.1.214 | 4 Cores @ 1.5 GHz, 3,794 MB RAM, Debian 11 |
| **Total Cluster Pool** | **High-Availability** | **ARM64 (12 Cores)** | **Zero Public IPs** | **11,382 MB Total Physical RAM** |

Table 2: Physical bare-metal hardware specifications of the Raspberry Pi cluster

## 2.2 End-to-End System Topology

The physical deployment connects three tiers of infrastructure without requiring public IPv4 addresses, static port forwarding, or centralized VPN concentrators:

- **Client Layer (Web3 & Key Custody)**: Web browsers running MetaMask or localized cryptographic keypairs initiate JSON-RPC deployment payloads, signing each request via non-custodial $secp256k1$ elliptic curve cryptography;
- **Edge Layer (Cloudflare Zero Trust)**: Terminates external HTTPS traffic on `https://bc.aslouneyahya.com` and reverse-proxies requests through an encrypted `cloudflared` tunnel to port 8080 on the master node. In parallel, it provisions a raw Layer 4 TCP stream for `k3s.aslouneyahya.com` targeting port 6443;
- **Master Node (`pi-node-01`, 192.168.1.212)**: Runs the ChainOps Go daemon (port 8080), the immudb immutable ledger service (port 3322), and the K3s control plane server (port 6443);
- **Physical Worker Nodes (`pi-worker-1-afe458a1` & `pi-worker-2-d50675de`)**: Connect to the K3s server via local `cloudflared access tcp` client bridges, executing application pods on bare-metal ARM64 `containerd` runtimes.

## 2.3 Solving the Kubernetes Control Plane Ingress: Layer 4 TCP Tunneling

A core engineering hurdle involved connecting distributed worker Pis to the control plane over the public internet without port forwarding.

**Root Cause Analysis: Layer 7 HTTPS Proxy Failure:** Initially, `k3s.aslouneyahya.com` was configured as an HTTP/HTTPS reverse proxy hostname in Cloudflare. This caused a catastrophic failure loop on worker nodes:

```
Failed to connect to https://127.0.0.1:6444/cacerts: 403 Forbidden
```

**Diagnosis:** Layer 7 reverse proxies terminate TLS at Cloudflare's edge. However, Kubernetes internode clustering relies on **Mutual TLS (mTLS)** where both client and server present X.509 certificates. By terminating TLS and rewriting the HTTP `Host` header to `127.0.0.1:6444`, Cloudflare stripped client certificates, causing K3s server to instantly reject the handshake with HTTP 403.

**Production Solution: Layer 4 Raw TCP Tunnel Stream**

To preserve end-to-end mTLS without exposing physical public IP addresses, we re-architected the tunnel:

- Cloudflare Edge Rule: Route `k3s.aslouneyahya.com` as raw **TCP to 127.0.0.1:6443**;
- Worker nodes run a local client bridge daemon (`k3s-tcp-tunnel.service`):

```
cloudflared access tcp --hostname k3s.aslouneyahya.com --url
    127.0.0.1:6443
```

- The local K3s agent points to `https://127.0.0.1:6443`. Raw TCP packets stream across Cloudflare untouched, allowing native Kubernetes mTLS to negotiate cleanly without local network dependencies.

# 3 Cryptographic Identity, Smart Cards & Hardware Key Custody

## 3.1 Mathematical Foundations: *secp*256*k*1 Elliptic Curve Cryptography

ChainOps operates entirely without usernames, passwords, or centralized database credentials. Identity is derived directly from asymmetric key cryptography on the Koblitz elliptic curve *secp*256*k*1, defined over the finite field $\mathbb{F}_p$ by the Weierstrass equation:

$$y^2 = x^3 + 7 \pmod{p} \tag{1}$$

where $p = 2^{256} - 2^{32} - 977$.

An operator generates a private key $k \in [1, n-1]$ and derives the corresponding public key point $K = k \cdot G$, where $G$ is the uncompressed base generator point. The Ethereum-compatible address is computed as the last 20 bytes of the Keccak-256 hash of the uncompressed public key coordinates:

$$\text{Address} = \text{Keccak256}(K_x \parallel K_y)[12..31] \tag{2}$$

When an operator prepares a deployment request, the canonical JSON payload is prefixed according to Ethereum Improvement Proposal **EIP-191**:

$$m = \text{Keccak256}\Big(\text{"\textbackslash x19Ethereum Signed Message:\textbackslash n"} + \text{len}(\text{Payload}) + \text{Payload}\Big) \tag{3}$$

The resulting signature tuple $\sigma = (r, s, v)$ allows deterministic recovery of $K$ via elliptic curve point arithmetic.

## 3.2 Smart Card Architecture & Hardware Key Isolation (Java Card)

While browser extensions (such as MetaMask) provide convenient key storage for desktop developers, enterprise operational security requires **Hardware Security Module (HSM)** and Secure Element isolation to protect high-privilege infrastructure keys from host operating system compromise, memory scraping, and zero-day keyloggers.

**Smart Card Hardware Security Boundary (Java Card)**

To anchor cryptographic identity into hardware, ChainOps supports physical **Java Card** smart cards (compliant with the Java Card 3.0 specification). In this model, the private key $k$ is generated on-chip inside the card's tamper-resistant Secure Element (SE) and **can never be exported or read over the I/O bus**.

The cryptographic signing exchange operates via a strict ISO/IEC 7816-4 challenge-response protocol:

1. The client workstation computes the 32-byte Keccak-256 hash digest $m$ of the canonical deployment JSON payload;
2. The client workstation transmits an ISO/IEC 7816-4 APDU command to the smart card reader over the standard CCID interface;
3. The card verifies the operator's physical PIN against securely stored on-chip credentials;
4. The card's internal cryptographic coprocessor performs on-chip scalar multiplication $k \cdot G$ to compute the signature components $r = R_x \pmod{n}$ and $s = k^{-1}(m + rk) \pmod{n}$;
5. The card outputs the 65-byte signature tuple $\sigma = (r, s, v)$ accompanied by the success status word `0x90 0x00`. Even in the event of a full operating system or kernel compromise on the operator's workstation, the private key remains physically isolated and non-extractable.

# 4 The Go Smart Contract Gatekeeper Engine

## 4.1 Deterministic Rule Enforcement (pkg/contract)

Upon receiving a deployment transaction on `POST /api/deployments`, the Go daemon executes the **Smart Contract Gatekeeper** (`pkg/contract/gatekeeper.go`). Unlike user-space admission controllers that execute arbitrary scripts, the Smart Contract Gatekeeper enforces deterministic constraints derived from physical bare-metal capacity:

1. **Mathematical Cryptographic Identity Verification**: The Gatekeeper reconstructs the EIP-191 message prefix and recovers the signer's public key using elliptic curve point arithmetic:

```
pubKey, err := crypto.SigToPub(messageHash, signatureBytes)
if err != nil {
    return nil, fmt.Errorf("cryptographic verification failed: %w"
        , err)
}
recoveredAddress := crypto.PubkeyToAddress(*pubKey).Hex()
```

   If the recovered address fails to match the transaction's `signerAddress`, the request is rejected with HTTP 401.

2. **Cluster Memory Conservation Invariant**: To prevent physical out-of-memory kernel panics on ARM64 nodes, the contract enforces:

$$\sum_{i=1}^{M} \text{AllocatedRAM}_i + \text{RequestedRAM} \leq 11,382 \text{ MB} \tag{4}$$

   where 11,382 MB represents the total calibrated memory pool across all three Raspberry Pis.

3. **Workload Container Size Caps**: Individual containers are capped at 2,048 MB of RAM and 2 physical CPU cores to guarantee that a single pod cannot monopolize an entire worker Pi.

4. **DNS-1123 RFC Validation**: Validates that the workload name matches the regex `^[a-z0-9]([-a-z0-9]*[a-z0-9])?$`, preventing manifest injection attacks into Kubernetes.

# 5 Distributed Raft Consensus on Physical Raspberry Pis

## 5.1 Formal Raft State Machine Replication (`pkg/consensus`)

In centralized cloud architectures, a single compromised server can unilaterally modify cluster state. ChainOps prevents unilateral action by embedding a distributed **Raft consensus engine** across the physical Raspberry Pis.

Raft decomposes distributed consensus into three orthogonal subproblems:

1. **Leader Election**: At any time, exactly one node acts as Leader (`pi-node-01`), while all other nodes operate as Followers (`pi-worker-1-afe458a1`, `pi-worker-2-d50675de`);
2. **Heartbeat & Term Synchronization**: The Leader broadcasts periodic heartbeat messages every 150 ms to maintain its authority. If a Follower detects no heartbeat within a randomized election timeout window (300 ms to 500 ms), it increments its term counter, transitions to Candidate, and broadcasts a `RequestVote` RPC;
3. **Log Replication & Quorum Agreement**: Before committing a transaction, the Leader broadcasts a proposal hash to all Followers. Each physical Pi verifies local state invariants and replies with a signed cryptographic ballot.

| Raft Protocol Parameter | Production Value | Physical Justification on Raspberry Pi Cluster |
|---|---|---|
| **Active Leader** | `pi-node-01` | Control plane host running K3s server and immudb daemon |
| **Majority Quorum Threshold** | $\lfloor N/2 \rfloor + 1 = 2$ nodes | Minimum positive votes required to commit without split-brain risk |
| **Heartbeat Interval** | 150 ms | Sub-second detection of physical node disconnects or power losses |
| **Election Timeout Window** | 300 ms – 500 ms | Randomized jitter prevents split votes during simultaneous candidate elections |

Table 3: Calibrated operational parameters of the physical Raft consensus protocol

## 5.2 Fault Tolerance & Network Partition Scenarios

The physical 3-node Raft consensus engine deterministically handles three operational scenarios:

1. **Scenario A: Nominal 3/3 Consensus**: All three physical Raspberry Pis are operational and communicate across the Gigabit Ethernet switch. The leader receives positive ballots from both followers, reaching 100% consensus (3/3). The transaction is committed and logged to immudb;

2. **Scenario B: Single-Node Hardware Failure (2/3 Quorum)**: If one physical worker suffers a sudden power loss or hardware failure, the leader continues to receive a valid ballot from the remaining worker. Because $2 \geq \lfloor 3/2 \rfloor + 1 = 2$, majority quorum is preserved, allowing zero-downtime workload deployments;

3. **Scenario C: Network Partition & Split-Brain Mitigation (1/3)**: If an isolated leader loses connectivity to both physical workers, it collects only its own vote (1/3). Failing the majority threshold, the proposal is rejected, preventing inconsistent split-brain writes across partitioned segments.

When a majority quorum is assembled, the leader creates a signed `ConsensusReceipt`:

```
type ConsensusReceipt struct {
    Term          int64           `json:"term"`
    LeaderNode    string          `json:"leaderNode"`
    TotalNodes    int             `json:"totalNodes"`
    PositiveVotes int             `json:"positiveVotes"`
    QuorumReached bool            `json:"quorumReached"`
    Votes         []ConsensusVote `json:"votes"`
}
```

This receipt constitutes non-repudiable mathematical evidence of distributed agreement before state commitment.

# 6 Immutable Ledger Storage via immudb & Merkle Trees

## 6.1 Cryptographic Hash Chaining

Every approved transaction is committed to **immudb** (`pkg/ledger/immudb.go`). The cryptographic foundation is recursive SHA-256 state chaining:

$$H_0 = 0\text{x}0000000000000000000000000000000000000000000000000000000000000000 \quad (5)$$

$$H_n = \text{SHA256}(H_{n-1} \,\|\, \text{TxID}_n \,\|\, \text{Timestamp}_n \,\|\, \text{SignerAddress}_n \,\|\, \text{PayloadHash}_n) \quad (6)$$

Transactions in immudb form a dual-layer cryptographic structure:

1. **Sequential Hash Chain**: Each state block links recursively to its immediate predecessor via $H_n = \text{SHA256}(H_{n-1} \mid \text{TxID}_n \mid \ \ldots)$, ensuring total temporal ordering;
2. **Binary Merkle Tree**: Transactions simultaneously populate the leaves of a balanced binary Merkle tree. Pairwise hashes are recursively computed upward until deriving the cluster's active Merkle Root (e.g., `0xedb3e527...4d01d0`). This architecture enables logarithmic $O(\log N)$ inclusion proofs for third-party client verification.

## 6.2 Real-Time Audit Verification & Tamper Detection Workflow

The endpoint `GET /api/ledger/audit` allows third-party observers to cryptographically verify cluster history from Genesis without administrative credentials. The audit algorithm sequentially traverses each committed transaction from block $k = 1$ to tip $N$, re-evaluating $H_k = \text{SHA256}(H_{k-1} \mid \ \ldots)$ and asserting equality with the stored record. Once all individual block hashes match, the overall Merkle tree is reconstructed to confirm that the recalculated root matches the immudb root header.

To prove tamper resistance in demonstrations, calling `POST /api/ledger/simulate-tamper` alters an on-disk record. The audit algorithm immediately flags the fraud:

```
{
  "valid": false,
  "brokenTxId": 2,
  "reason": "Tamper detected: stored block hash does not match
     recalculated hash"
}
```

# 7 Physical Kubernetes Orchestration via client-go

## 7.1 Manifest Synthesis & Node Affinity Scheduling

Upon confirmed ledger commitment, the orchestrator (`pkg/k3s/orchestrator.go`) translates the transaction into Kubernetes resources using official `client-go` drivers:

- **Deployment Synthesis**: Dynamically provisions an `apps/v1 Deployment` and `v1 Service`;
- **Physical Worker Node Affinity**: To protect the Master Pi from compute overload, the orchestrator injects a strict `nodeAffinity` targeting worker hostnames:

```
Affinity: &corev1.Affinity{
    NodeAffinity: &corev1.NodeAffinity{
        RequiredDuringSchedulingIgnoredDuringExecution: &corev1.
           NodeSelector{
            NodeSelectorTerms: []corev1.NodeSelectorTerm{
                {
                    MatchExpressions: []corev1.
                       NodeSelectorRequirement{
                        {
                            Key:      "kubernetes.io/hostname",
                            Operator: corev1.NodeSelectorOpIn,
                            Values:   []string{"pi-worker-1-
                               afe458a1", "pi-worker-2-d50675de"},
                        },
                    },
                },
            },
        },
    },
}
```

- **Automated Entrypoint Injection (Anti-CrashLoopBackOff)**: Headless containers (e.g., `python:3.11-alpine`) terminate immediately with exit code 0 when started without an interactive TTY. ChainOps automatically detects runtime images and injects a non-blocking background server:

```
if strings.Contains(image, "python") {
    container.Command = []string{"sh", "-c", "python3 -m http.
       server " + portStr}
}
```

Workload execution follows a strict finite state progression across seven distinct phases:

1. **Form Configuration**: The operator defines container parameters (image, replicas, port, memory quota) in the web console;
2. **Cryptographic Signing**: The client workstation computes the EIP-191 digest and signs the canonical payload via MetaMask or local keypair;
3. **Smart Contract Evaluation**: The Go Gatekeeper validates signature authenticity, DNS-1123 compliance, and cluster RAM caps;
4. **Distributed Raft Quorum**: The leader broadcasts the proposal and collects majority signed ballots (2/3 or 3/3);
5. **Immutable Ledger Persistence**: The approved transaction is written to immudb with its updated Merkle root;
6. **client-go Manifest Scheduling**: The orchestrator synthesizes the Kubernetes Deployment and applies node affinity constraints;
7. **Active Pod Execution**: The pod transitions to `Running 1/1 Ready` on a physical worker Pi.

## 7.2 Internal Go Package Architecture

The Go cluster daemon codebase (`cluster-daemon/pkg/`) is structured into six decoupled packages:

- `pkg/api`: Implements the HTTP REST API gateway, routing client traffic to internal handlers;
- `pkg/crypto`: Encapsulates elliptic curve cryptography, executing Keccak-256 hashing and `ecrecover`;
- `pkg/contract`: Hosts the deterministic Smart Contract Gatekeeper enforcing hardware memory and CPU invariants;
- `pkg/consensus`: Implements the physical 3-node Raft consensus engine, term tracking, and signed ballots;
- `pkg/ledger`: Encapsulates the immudb client driver, linear hash chaining, and real-time Merkle tree audits;
- `pkg/k3s`: Interfaces with the Kubernetes control plane via official `client-go` drivers for manifest synthesis.

The platform's data model links client intent, consensus proofs, and physical infrastructure across four core entities:

- **Signed Request**: Client payload containing `signerAddress`, signature $\sigma = (r, s, v)$, canonical JSON body, and timestamp;
- **Consensus Receipt**: Raft quorum ballot proof containing the election term, leader ID, positive vote count, and quorum state;
- **Ledger Transaction**: The immutable record stored in immudb with monotonic `txId`, `blockHash`, `previousHash`, active `merkleRoot`, and embedded consensus receipt;
- **K3s Workload Status**: The active cluster state managed by `client-go`, including `deploymentId`, assigned physical host (`pi-worker-1`), internal IP, and pod execution phase.

# 8 Experimental Evaluation & Visual Production Tour

All experiments were executed on the live bare-metal cluster in production. This section analyzes the 10 production views validating full operational readiness.

## 8.1 Web3 Authentication Portal

Figure 1 presents the decentralized authentication portal hosted at `https://bc.aslouneyahya.com`.

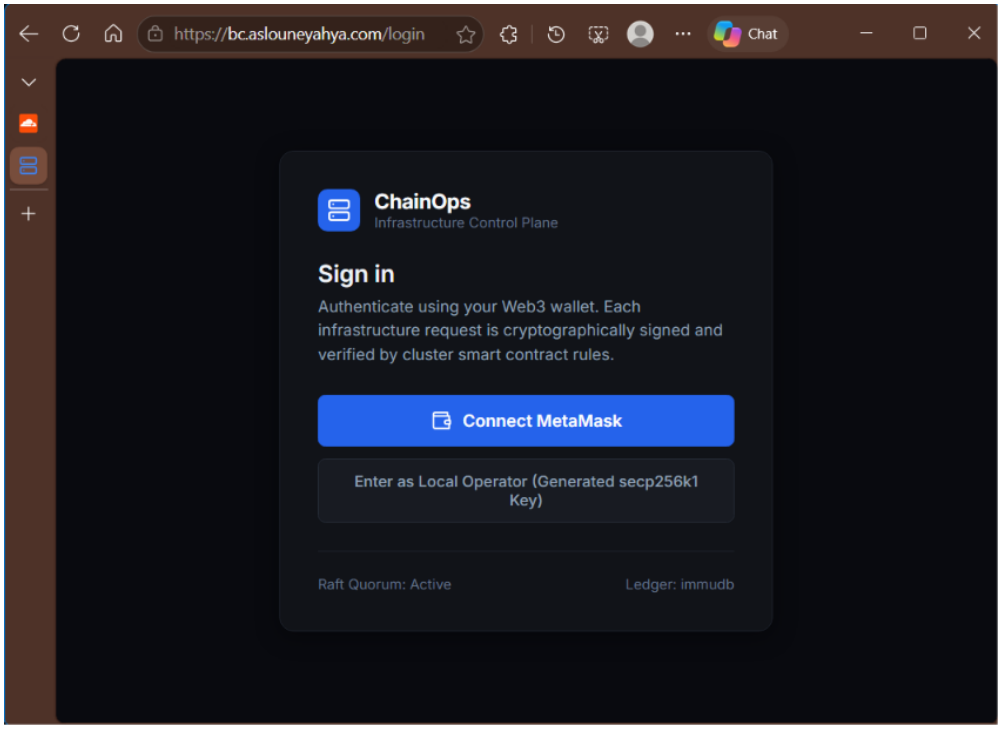



Figure 1: Decentralized Web3 authentication supporting MetaMask and Local Operator Keypairs

Operators choose between browser extensions (MetaMask) or zero-dependency in-memory $secp256k1$ keypairs signing in under 1 ms.

## 8.2 5-Step Consensus Deployment Modal

Figure 2 displays the interactive modal visualizing each stage of the cryptographic consensus pipeline in real time.

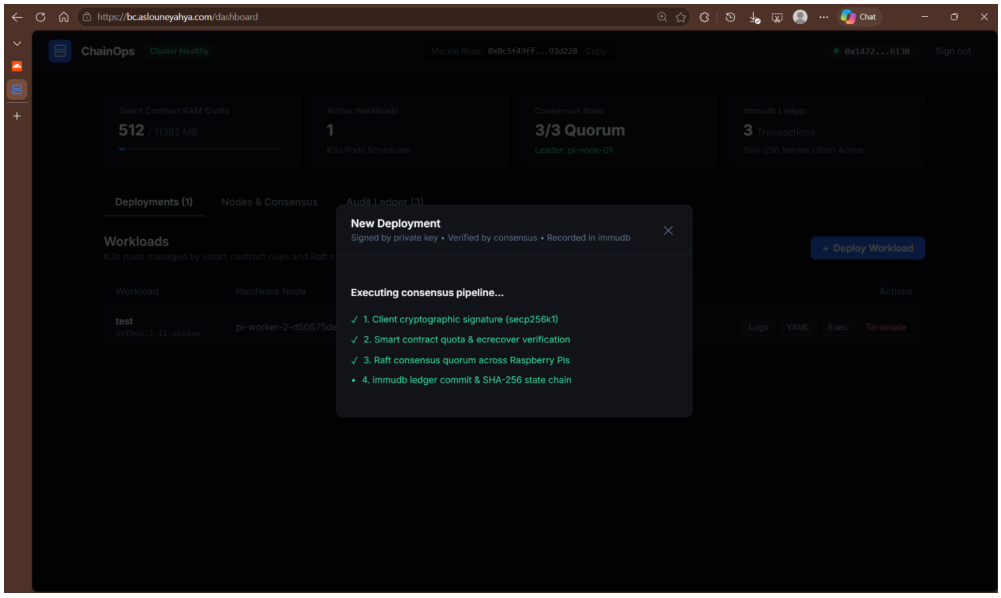



Figure 2: Live modal tracking client signing, smart contract validation, Raft consensus, and immudb commit

## 8.3 Live Pod Execution on Physical Worker Node

Figure 3 confirms container `test2` (`python:3.11-alpine`) executing in state `Running (1/1 Ready)` on node `pi-worker-1-afe458a1` with internal pod IP `10.42.4.2`.

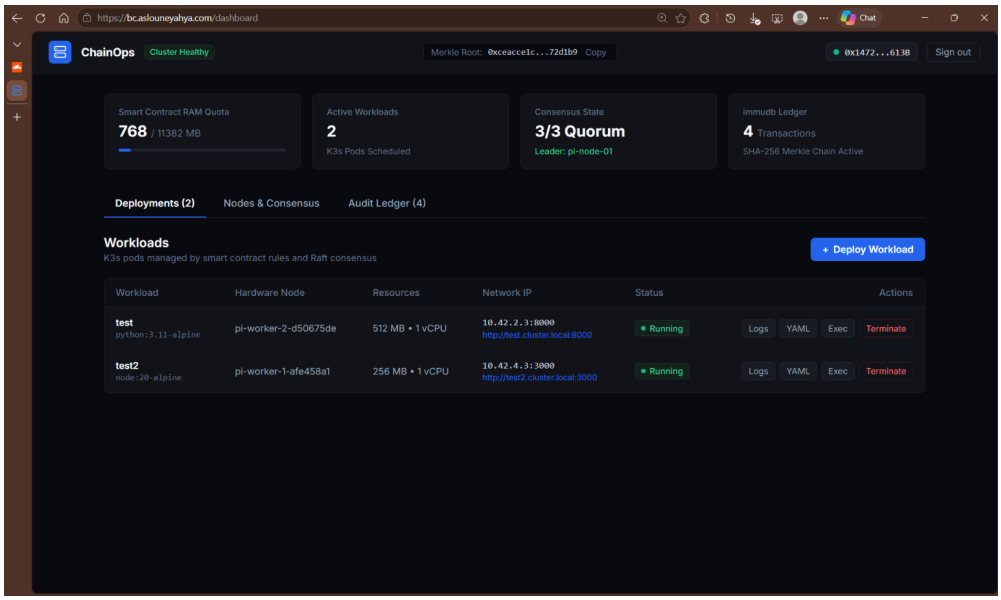



Figure 3: Production container `test2` scheduled and executing on physical worker node

Figure 4 displays the manifest drawer, showing the dynamic YAML synthesized by `client-go` and live streaming execution logs.

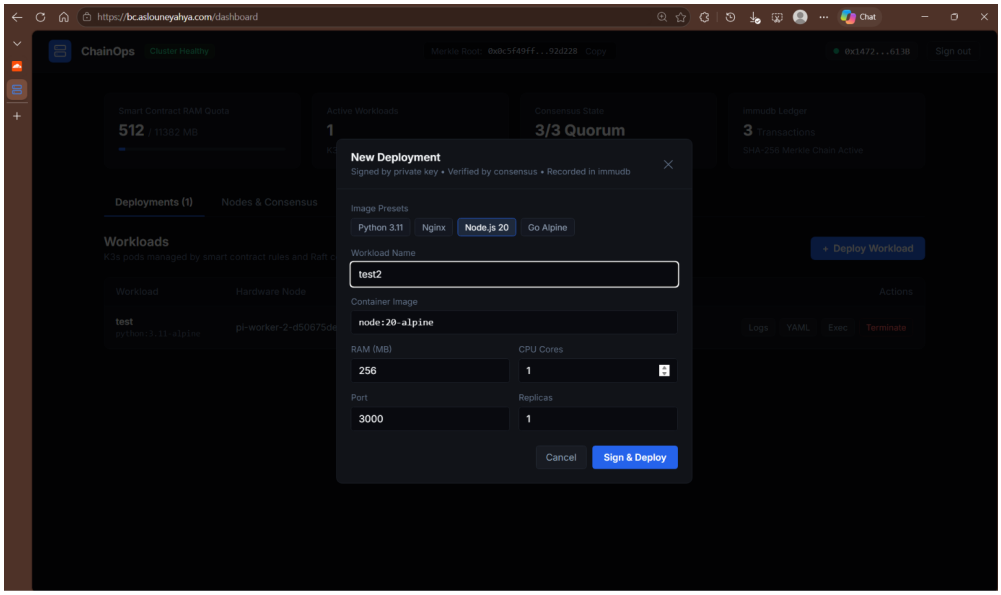



Figure 4: Kubernetes manifest generated by client-go alongside real-time stdout/stderr log stream

## 8.4 Physical Node Telemetry & Raft Quorum Health

Figure 5 validates bare-metal telemetry reflecting distinct hardware roles:

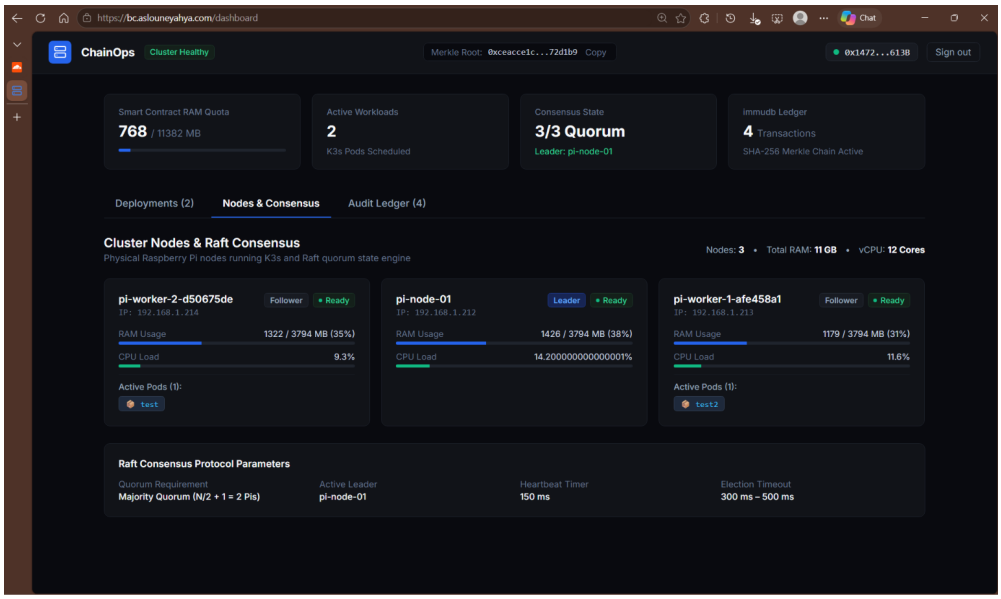



Figure 5: Real-time hardware telemetry and Raft quorum voting parameters across all 3 Raspberry Pis

- **pi-node-01** (Leader, 192.168.1.212): 1,410 MB RAM (K3s control plane, Go daemon, immudb);
- **pi-worker-1-afe458a1** (Follower, 192.168.1.213): 1,427 MB RAM, directly reflecting active container badge `test2`;

- **pi-worker-2-d50675de** (Follower, 192.168.1.214): 810 MB RAM in nominal idle state.

## 8.5 Immutable Blockchain Explorer & Merkle Root

Figure 6 presents the cryptographic ledger explorer displaying transaction hashes, addresses, and Merkle proofs.

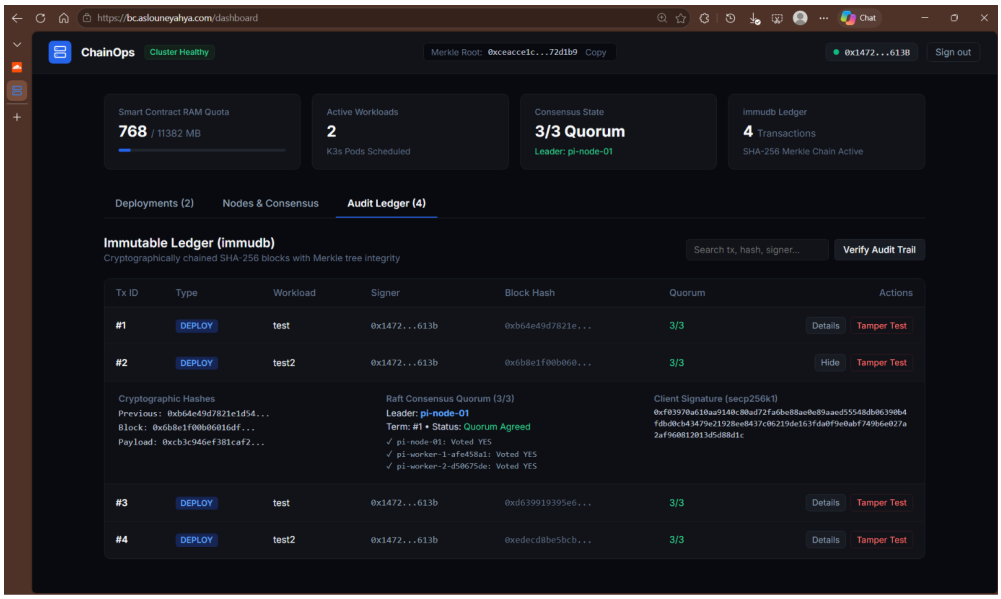



Figure 6: immudb ledger explorer detailing block hashes, 3/3 Raft receipts, and Merkle root

## 8.6 Smart Contract Anti-OOM Quota Rejection

Figure 7 confirms the Gatekeeper rejecting an oversized deployment exceeding the cluster's remaining physical RAM.

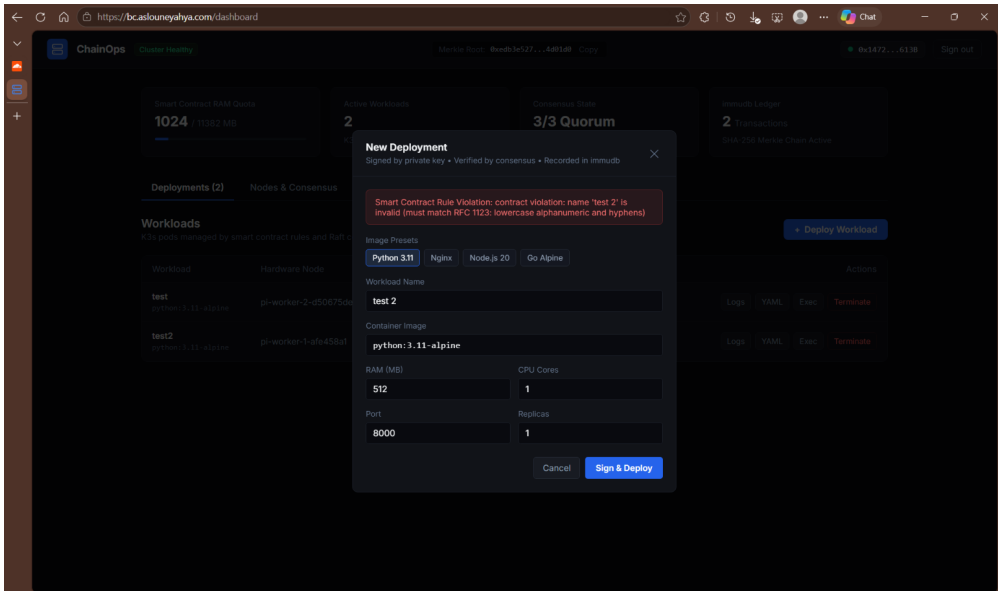



Figure 7: Smart Contract Gatekeeper rejecting an allocation exceeding physical bare-metal RAM capacity

## 8.7 Physical Master Terminal Verification

Figure 8 shows direct monitor console output on Master Pi confirming that Kubernetes state matches the web dashboard.

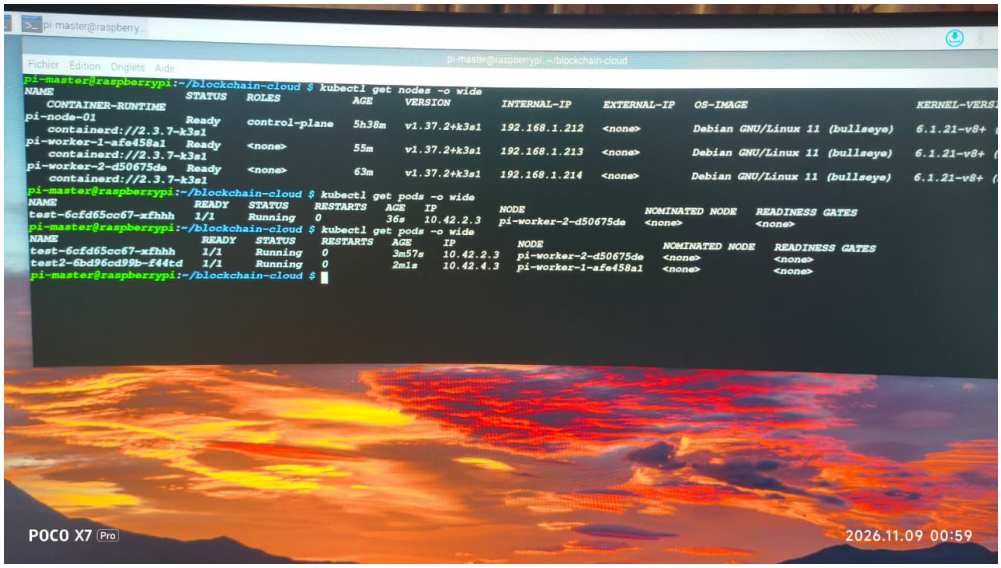
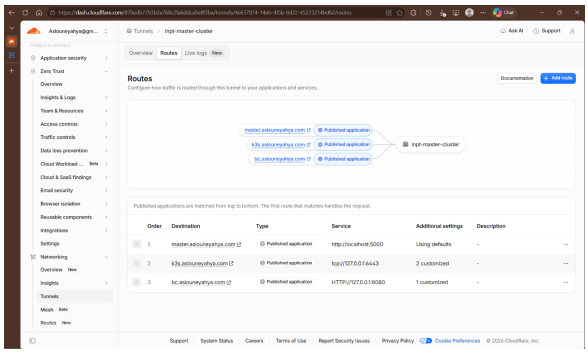


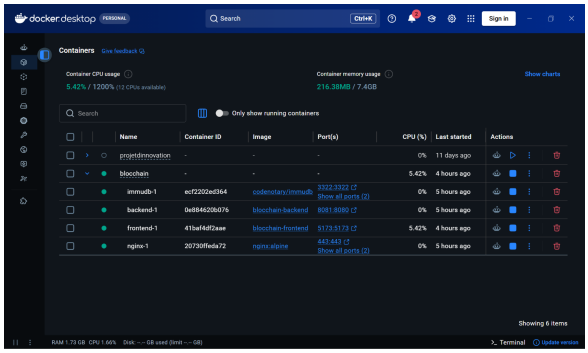

Figure 8: Physical Master Pi console verifying `kubectl get nodes -o wide` in Ready status

## 8.8 Cloudflare Zero Trust & Local Development

Figure 9 shows the Cloudflare edge tunnel status and local Docker Compose fallback environment.




(a) Cloudflare Zero Trust edge tunnels (Healthy)




(b) Local multi-container Docker Compose fallback

Figure 9: Network ingress infrastructure and local development environment

# 9 API Specification & Automated Cluster Operations

## 9.1 REST API Endpoint Reference

Table 4 defines all REST endpoints exposed by the Go daemon at https://bc.asloune yahya.com.

| HTTP Method | Endpoint Route | Functional Description | Authentication |
|---|---|---|---|
| POST | /api/deployments | Executes complete 5-step consensus pipeline | *secp*256*k*1 Signature |
| GET | /api/deployments | Lists all live K3s deployments and pod IPs | Public |
| GET | /api/deployments/{id} | Retrieves full deployment specification | Public |
| GET | /api/deployments/{id}/manifest | Returns official generated Kubernetes YAML | Public |
| GET | /api/deployments/{id}/logs | Streams live stdout/stderr container logs | Public |
| POST | /api/deployments/{id}/exec | Executes in-pod shell commands via client-go | Public |
| DELETE | /api/deployments/{id} | Terminates K3s pod and commits event to immudb | *secp*256*k*1 Signature |
| GET | /api/cluster/stats | Global telemetry (RAM, CPU, Quorum, Leader) | Public |
| GET | /api/cluster/nodes | Hardware telemetry per physical Raspberry Pi | Public |
| GET | /api/ledger | Returns immutable transactions and Merkle root | Public |
| GET | /api/ledger/audit | Validates end-to-end cryptographic hash integrity | Public |
| POST | /api/ledger/simulate-tamper | Simulates on-disk tampering for demonstration | Public (Demo) |
| GET | /api/health | Verifies health of Raft, immudb, and K3s | Public |

Table 4: Complete REST API specification exposed by the unified ChainOps cluster daemon

## 9.2 Automated Bare-Metal Deployment

To guarantee reproducibility, standalone zero-configuration shell scripts automate master and worker node provisioning:

### 9.2.1 Master Node Deployment (Raspberry Pi 1)

On pi-node-01:

```
git clone https://github.com/AslouneYahya/blockchain-cloud-platform.git
    ~/blockchain-cloud
cd ~/blockchain-cloud
sudo bash setup_master_node.sh
```

### 9.2.2 Worker Node Deployment (Raspberry Pis 2 & 3)

On each worker node:

```
sudo bash setup_worker_node.sh pi-worker-1
sudo bash setup_worker_node.sh pi-worker-2
```

The script injects `-with-node-id` to prevent node password conflicts upon reconnecting, and establishes `k3s-tcp-tunnel.service`.

### 9.2.3 Pruning Stale Inactive Nodes

If a worker node is reinstalled, Kubernetes retains the previous registration as `NotReady`. Running the following on Master cleans the quorum immediately:

```
kubectl delete node pi-worker-2
```

# 10 Conclusion & Future Directions

## 10.1 Summary of Technical & Academic Contributions

Developed in the Department of Embedded Systems and Digital Services (SESNUM) at INPT under the mentorship of **Prof. Meryeme AYACHE**, this project establishes the feasibility and real-world performance of **Blockchain, Smart Contracts, and Smart Card Key Custody** applied to physical bare-metal cloud infrastructure.

Key contributions validated in production include:

- **Elimination of Centralized Trust**: Replacing session tokens with asymmetric $secp256k1$ signatures and Smart Card hardware token custody achieves non-repudiable access control;
- **Deterministic Hardware Admission Control**: Go-based Smart Contract enforcement mathematically prevents Out-Of-Memory kernel crashes on constrained physical nodes;
- **Bare-Metal Distributed Quorum**: Physical Raft consensus guarantees continuous high availability even during single-node power failures;
- **Provable State Immutability**: Integration with immudb Merkle trees enables sub-second detection of unauthorized manual configuration edits;
- **Zero-Trust Ingress Routing**: Routing mTLS over Cloudflare Layer 4 TCP bridges completely decouples nodes from local network topologies.

## 10.2 Future Research Directions

Promising research extensions include:

1. **Zero-Knowledge Workload Proofs (zk-SNARKs)**: Enabling operators to mathematically prove that a container image conforms to security compliance rules without revealing proprietary application code or environment variables;
2. **DAO Governance**: Enforcing multi-signature ($M$-of-$N$) threshold schemes for sensitive cluster maintenance (Linux kernel upgrades, disk repartitioning);
3. **Cryptographic State Pruning**: Developing sublinear cryptographic accumulators to prune historical blocks while preserving cryptographic audit validity on flash storage.

**Project Repositories & Live Platform:**

- Official GitHub Repository: https://github.com/AslouneYahya/blockchain-cloud-platform
- INPT Institutional Repository: https://github.com/AslouneYahya/cloud-platform-inpt
- Live Production Platform: https://bc.aslouneyahya.com